

IPAM CONNECT

Student Networking & Communication Platform

Document Type	Product Requirement Document (PRD)	Version	1.1.0 (Production Grade)
Target Platform	Vercel (Serverless Edge)	Date Created	June 10, 2026
Author Team	Full-Stack Creative Team	Status	Approved & Baseline Locked

1. EXECUTIVE SUMMARY & PROBLEM STATEMENT

1.1 Problem Statement

Students at IPAM often encounter significant friction when attempting to network, collaborate, and exchange knowledge outside formal classroom physical infrastructure. Academic and social communication remains highly fragmented across multiple commercial social media applications. This systemic fragmentation prevents optimal peer-to-peer resource distribution, coordinates academic collaboration inefficiently, and severely isolates professional networking potential within the student body.

1.2 Proposed Solution

IPAM CONNECT is a secure, institutional networking web application custom-engineered for modern campus realities. The web platform enables students to form authenticated identity records, access unified course directories structured around official enrollment parameters, utilize seamless messaging channels, and exchange rich learning materials securely and reliably within a standardized ecosystem.

2. USER PERSONAS & SYSTEM ROLES

The platform recognizes three operational actors with tiered permission logic across all workspaces:

- **System Administrator (Admin):** Manages core platform parameters, configures database entries for module structures, provisions student leadership clearance statuses, and posts to the global read-only Announcement Feed.
- **Class Representative (Class Rep):** Institutional community moderators designated for specific course channels. Class Reps retain exclusive capabilities to inspect content, delete non-compliant message data, clear corrupted files, and uphold community standards.
- **Student:** Verified users authorized to view student registries, perform granular department queries, engage in collaborative chats, communicate via direct channels, and distribute learning attachments.

3. ARCHITECTURE & CLOUD INFRASTRUCTURE SPEC

To support serverless operational constraints and enable elastic scaling under heavy concurrent usage, the stack is configured according to strict decoupled data patterns:

Layer	Technology	Architectural Purpose
Frontend Framework	Next.js (App Router), Tailwind CSS	Handles server-side rendering (SSR), structural route protection, and presents a responsive, mobile-first utility layout.
Backend Compute	Next.js API Routes	Executes stateless backend code blocks to process input variables, handle secure authorization checks, and orchestrate third-party API payloads.
Primary Database	MongoDB	Serves as the foundational schema store for student accounts, community channel groups, and transactional resource records.
Real-Time Pipeline	Pusher Channels (Serverless Pub/Sub)	Manages stateful, continuous client connections externally. Bypasses serverless timeout constraints to allow instant data broadcasting.
Distributed Storage	AWS S3 (Simple Storage Service)	Acts as the highly durable file repository for multi-format academic uploads, separated cleanly from main compute layers.
Identity Provider	NextAuth.js (Auth.js)	Orchestrates standardized JSON Web Token (JWT) handling and manages OAuth 2.0 flows via Google and GitHub sign-in vectors.
Deployment Target	Vercel Infrastructure	Hosts the global deployment, offering edge optimization and fast automated build-pipelines.

4. FUNCTIONAL SPECIFICATIONS & FEATURE MECHANICS

4.1 User Authentication & Onboarding Flow

- **Unified Sign-On:** General authentication allows registration from any active Gmail account or supported provider via NextAuth.js.

- **Mandatory Profile Onboarding:** Upon initial sign-on, a strict redirection wall intercepts the session. Users must submit a complete, single-submission onboarding entry before accessing any system features.

Required attributes:

- Full Name
 - Student ID Number (*Protected by a unique system constraint check to eliminate duplicate account claims*)
 - Department / Major Focus
 - Current Academic Year
- **Concurrency Scope:** Active user sessions are tracked via standard persistent authorization tokens, allowing a student to interact via the same Gmail profile across multiple physical devices simultaneously.

4.2 Student Directory & Privacy Control

- **Indexed Search Index:** Contains profiles of all registered students, allowing directory results to be filtered by **Department** and **Year of Study**.
- **Granular Privacy Toggles:** Individual user profiles feature access toggles, allowing students to mask detailed variables (e.g., direct email fields) from general non-connected directory queries.

4.3 Channel Spaces & Announcement Feed

- **Course Groups:** Dedicated read/write communication environments. To maintain order, these spaces are strictly pre-generated by System Admins using official university curricula.
- **Global Announcement Feed:** High-priority global notice panel. This feed operates under a strict read-only model for general student tiers; only authorized System Admins and Student Leaders possess creation privileges.

4.4 Real-Time Communication Pipeline

To avoid managing manual, stateful TCP pipelines on headless serverless infrastructure, real-time message exchange relies entirely on the integrated Pusher micro-service layer.

Multi-Device State Management Matrix:

When an action triggers a database write from Device A, the stateless API route fires a companion update payload to the Pusher HTTP gateway. The client application utilizes unique, secure channels to update active states concurrently across all active user views:

- **Direct Messages:** Routed on individual, dual-delivery tracks mapped to specific peer channels.
- **Group Modules:** Handled through unified presence channel strings matching MongoDB course entity tracking keys (e.g., `presence-course-[courseId]`).

4.5 File Distribution Engine & S3 Presigned URL Architecture

To prevent payload size faults and comply with Vercel's strict serverless request limitation of 4.5MB, **no binary data chunks are ever passed through or parsed by the local Next.js environment**. All operations utilize secure presigned client-to-storage bypass paths.

Step-by-Step Presigned Upload Flow:

1. The user targets an asset file from their device interface (`<50 MB>`) and initiates submission.
2. The browser issues a lightweight structural JSON metadata request to the endpoint `/api/resources/create-upload-url` detailing file name, extension parameters, and destination group identifier.
3. The backend validates active session state, checks extension compliance against permitted file formats (`.pdf`, `.docx`, `.png`, `.jpg`, `.jpeg`, `.zip`), and requests a cryptographically signed **S3 PUT URL** with a 5-minute expiration block via the AWS SDK.
4. The signed path string returns to the frontend client.
5. The browser executes an unproxied HTTP PUT request, shifting raw file binary data **directly into the cloud AWS S3 Bucket storage container**.
6. Upon receiving a successful `200 OK` from the S3 storage node, the client executes a final record sync to write the asset index metadata back to MongoDB, dynamically mapping it to the active Course Group window.

Infrastructure Abuse Safeguards: Individual profiles are bound by a rigid hardware infrastructure storage ceiling. Each student record is allocated a cumulative maximum storage pool of `500 MB`. Write permissions are suspended if the sum of a user's logged files in MongoDB breaches this limit.

4.6 Notification Pipeline

Real-time, reactive UI toast alerts and red notification badges are triggered instantly via WebSockets across three strict contexts:

1. Incoming 1-on-1 Direct Messages.
2. New posts or messages published within a Course Group the student belongs to.
3. Global, administrative postings published to the Announcement Feed.

5. MODERATION, GOVERNANCE & ACCESS SECURITY

- **Decentralized Content Pruning:** Group moderation is delegated down to Class Representatives. They can access special channel controls to instantly remove non-compliant discussions or clean problematic media files.
- **Structural Security Guards:** Rigorous JSON and BSON schema constraints are applied inside the backend database abstraction layers to insulate parameters from cross-site scripting (XSS) input attacks, script injections, or document payload manipulation.

6. NON-FUNCTIONAL TECHNICAL BENCHMARKS (NFRS)

- **Interface Target Latency:** UI component switches, message render responses, and presigned coordinate allocation must stay within sub-*150 ext{ms}* thresholds under normal network conditions.
- **Optimistic Write Concurrency:** To manage rapid multi-device user inputs without locking data collections, database modifications are protected by versioning state keys (`__v`) using optimistic concurrency controls.
- **Network Resilience:** Upload modules must include automatic retry-on-failure wrappers to smooth over unexpected transitions between campus Wi-Fi access networks and mobile cellular coverage data channels.